

LAPORAN PRAKTIKUM
STRUKTUR DATA
BINARY TREE DAN GRAPH



OLEH:
DEVINA AMANDA PUTRI
(2411533009)

DOSEN PENGAMPU:
DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2024

A. PENDAHULUAN

Binary Tree atau pohon biner adalah struktur data di mana setiap simpul (node) hanya memiliki maksimal dua anak, yaitu kiri (left) dan kanan (right). Struktur ini sering digunakan dalam berbagai aplikasi seperti kompresi data (Huffman Coding), parsing ekspresi, hingga implementasi struktur data efisien seperti Binary Search Tree (BST). Dalam pohon biner, traversal atau penelusuran dapat dilakukan melalui tiga cara utama: preorder, inorder, dan postorder, masing-masing memiliki urutan kunjungan node yang berbeda.

Selain pohon biner, praktikum ini juga memperkenalkan konsep dasar graf melalui simulasi graf tak berarah dan implementasi algoritma penelusuran menggunakan DFS (Depth-First Search) dan BFS (Breadth-First Search). Penelusuran pada graf sangat penting dalam berbagai bidang seperti perutean jaringan (network routing), sistem rekomendasi, hingga pencarian jalur terpendek. Dengan menggabungkan pemahaman tentang tree dan graph traversal, mahasiswa diharapkan mampu memahami struktur data non-linear dan cara menelusurnya secara efisien dalam bentuk implementasi Java.

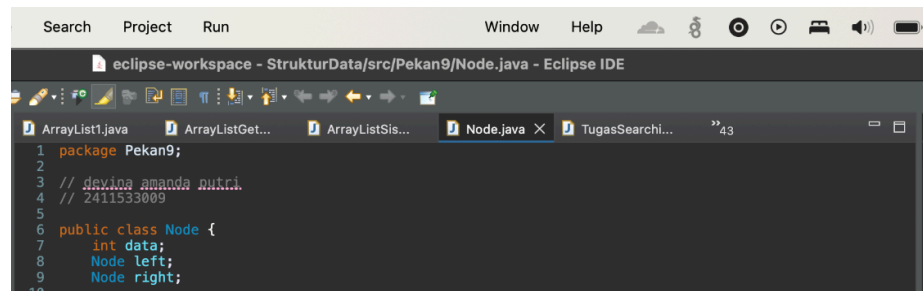
B. TUJUAN

1. Memahami algoritma struktur data pohon biner dan graf
2. Mengimplementasikan algoritma tersebut dalam program Java

C. LANGKAH-LANGKAH Pengerjaan

a. Node.java

1. Buat class baru pada package Pekan9 dan beri nama Node.java. Deklarasikan variabel untuk menyimpan data atau nilai dari node. Tambahkan pointer ke anak kiri dan anak kanan dari node saat ini



```
1 package Pekan9;
2
3 // dayana amanda nuzul
4 // 2411533009
5
6 public class Node {
7     int data;
8     Node left;
9     Node right;
10 }
```

2. Membuat konstruktor yang menginisialisasikan data dan menyetel anak kiri dan kanan menjadi null



```
10
11 public Node(int data) {
12     this.data = data;
13     left = null;
14     right = null;
15 }
16
```

3. Buat method untuk mengatur anak kiri, tapi hanya jika belum ada anak kiri sebelumnya



```
15
16
17 public void setLeft(Node node) {
18     if (left == null)
19         left = node;
20 }
21
```

4. Lalu method untuk mengatur anak kanan, hanya jika belum ada sebelumnya.



```
21
22 public void setRight(Node node) {
23     if (right == null)
24         right = node;
25 }
26
```

5. Tambahkan getter untuk kiri dan kanan dan juga getter untuk nilai data node. Tambahkan juga setter untuk memperbarui data node



```
25
26
27 public Node getLeft() {
28     return left;
29 }
30
31 public Node getRight() {
32     return right;
33 }
34
35 public int getData() {
36     return data;
37 }
38
39 public void setData(int data) {
40     this.data = data;
41 }
42
```

6. Method rekursif untuk mencetak node dengan urutan preorder atau root - left - right

```
42
43 void printPreorder(Node node) {
44     if (node == null)
45         return;
46     System.out.print(node.data + " ");
47     printPreorder(node.left);
48     printPreorder(node.right);
49 }
50
```

Method rekursif untuk mencetak node dalam urutan postorder atau left - right - root

```
50
51 void printPostorder(Node node) {
52     if (node == null)
53         return;
54     printPostorder(node.left);
55     printPostorder(node.right);
56     System.out.print(node.data + " ");
57 }
58
```

Method rekursif untuk mencetak node dengan urutan inorder atau left - root - right

```
58
59 void printInorder(Node node) {
60     if (node == null)
61         return;
62     printInorder(node.left);
63     System.out.print(node.data + " ");
64     printInorder(node.right);
65 }
66
```

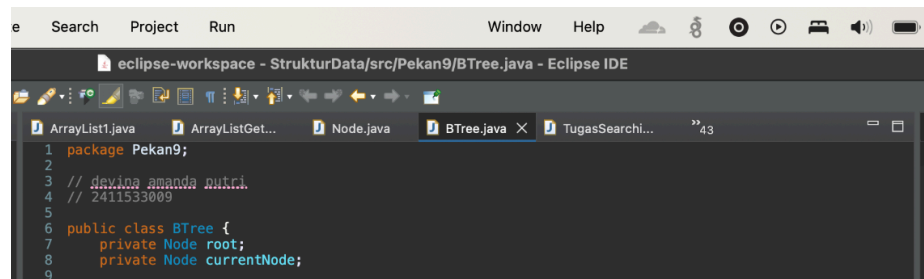
7. Buat method untuk mencetak visualisasi stuktur pohon, tambahkan juga method untuk membantu mencetak tree dalam format visual dengan garis cabang. Cetak node kanan terlebih dahulu agar tampil di atas saat divisualisasi dan cetak node saat ini dengan format cabang sesuai posisi, lalu cetak anak kiri setelah anak kanan supaya layout nya menyerupai struktur pohon vertikal

```
66
67 public String print() {
68     return this.print("", true, "");
69 }
70
71 public String print(String prefix, boolean isTail, String sb) {
72     if (right != null) {
73         right.print(prefix + (isTail ? "| " : " "), false, sb);
74     }
75     System.out.println(prefix + (isTail ? "└─ " : "┌─ ") + data);
76     if (left != null) {
77         left.print(prefix + (isTail ? " " : "| "), true, sb);
78     }
79     return sb;
80 }
81 }
82
```

b. BTree.java

1. Buat class baru pada package yang sama, deklarasikan class BTree yang merepresentasikan pohon biner, root sebagai akar dan

currentNode adalah node yang sedang aktif



```
1 package Pekan9;
2
3 // devina amanda nurri.
4 // 2411533009
5
6 public class BTree {
7     private Node root;
8     private Node currentNode;
9 }
```

2. Tambahkan konstruktor untuk menginisialisasi pohon dengan root kosong.



```
9
10 public BTree() {
11     root = null;
12 }
13
```

3. Method publik untuk mencari nilai dalam tree yang dimulai dari root, jika data ditemukan pada node ini, kembalikan ke true. Jika anak kiri tidak null, carilah data secara rekursif ke kiri, jika anak kanan tidak null maka cari data ke kanan dan jika data tidak ditemukan, kembalikan false



```
17
18 private boolean search(Node node, int data) {
19     if (node.getData() == data) {
20         return true;
21     }
22     if (node.getLeft() != null) {
23         if (search(node.getLeft(), data)) {
24             return true;
25         }
26     }
27     if (node.getRight() != null) {
28         if (search(node.getRight(), data)) {
29             return true;
30         }
31     }
32     return false;
33 }
```

4. Method publik untuk mencetak isi pohon dengan tiga jenis penelusuran.



```
29
30 public void printInorder() {
31     root.printInorder(root);
32 }
33
34 public void printPreorder() {
35     root.printPreorder(root);
36 }
37
38 public void printPostorder() {
39     root.printPostorder(root);
40 }
41
```

5. Buat getter untuk root dan pengecekan apakah pohon kosong. Lalu buat method untuk menghitung total simpul dalam tree secara rekursif, tampilkan tree secara visual. Getter/setter untuk currentNode dan root

```

41
42 public Node getRoot() {
43     return root;
44 }
45 public boolean isEmpty() {
46     return root == null;
47 }
48
49 public int countNodes() {
50     return countNodes(root);
51 }
52
53 private int countNodes(Node node) {
54     int count = 1;
55     if (node == null) {
56         return 0;
57     } else {
58         count += countNodes(node.getLeft());
59         count += countNodes(node.getRight());
60         return count;
61     }
62 }
63
64 public void print() {
65     root.print();
66 }
67
68 public Node getCurrent() {
69     return currentNode;
70 }
71
72 public void setCurrent(Node node) {
73     this.currentNode = node;
74 }
75
76 public void setRoot(Node root) {
77     this.root = root;
78 }
79 }

```

c. TreeMain.java

1. Buat class TreeMain pada package yang sama. Method TreeMain akan menjadi method utama yang dijalankan saat program dimulai

```

Search Project Run Window Help
eclipse-workspace - StrukturData/src/Pekan9/TreeMain.java - Eclipse IDE
ArrayList1.java Node.java BTree.java *TreeMain.java TugasSearchi...
1 package Pekan9;
2
3 // devina amanda putri
4 // 2411533009
5
6 public class TreeMain {
7     public static void main(String[] args) {

```

2. Buat pohon baru dan menampilkan jumlah node awal, membuat simpul pertama bernilai 1 dan menjadikan sebagai root lalu mencetak jumlah simpul

```

8
9 BTree tree = new BTree();
10 System.out.print("Jumlah simpul awal pohon: ");
11 System.out.println(tree.countNodes());
12
13
14 Node root = new Node(1);
15
16 tree.setRoot(root);
17 System.out.println("Jumlah simpul jika hanya ada root");
18 System.out.println(tree.countNodes());
19

```

3. Buat node tambahan yang bernilai 2 sampai 7

```
26
27     root.setLeft(node2);
28     node2.setLeft(node4);
29     node2.setRight(node5);
30     node3.setLeft(node6);
31     root.setRight(node3);
32     node3.setRight(node7);
33
```

4. Menyusun stuktur tree, lalu menyimpan node root sebagai currentNode dan tampilkan

```
root.setLeft(node2);
node2.setLeft(node4);
node2.setRight(node5);
node3.setLeft(node6);
root.setRight(node3);
node3.setRight(node7);
```

```
tree.setCurrent(tree.getRoot());
System.out.println("Menampilkan simpul terakhir: ");
System.out.println(tree.getCurrent().getData());
```

5. Cetak jumlah simpul setelah semua node ditambahkan

```
38     System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
39     System.out.println(tree.countNodes());
40
41
```

6. Cetak penelusuran tree dalam urutan inorder, preorder dan postorder, lalu tampilkan struktur pohon dalam bentuk visual

```
42     System.out.println("InOrder: ");
43     tree.printInorder();
44
45     System.out.println("\nPreOrder: ");
46     tree.printPreorder();
47
48     System.out.println("\nPostOrder: ");
49     tree.printPostorder();
50
51     System.out.println("\nMenampilkan simpul dalam bentuk pohon");
52     tree.print();
53 }
```

Cisco Packet Tracer 8.2.2

d. GraphTransversal.java

1. Buat class baru pada pacakge yang sama, import library Java Collection untuk Map, List, Set dll. Deklarasi class untuk penelusuran graf

```
1 package Pekan9;
2
3 //devina amanda putri
4
5 import java.util.*;
6
7 public class GraphTransversal {
8     private Map<String, List<String>> graph = new HashMap<>();
9 }
```

2. Menambahkan edge atau graf tak berarah anatar dua simpul

```
11 public void addEdge(String node1, String node2) {
12     graph.putIfAbsent(node1, new ArrayList<>());
13     graph.putIfAbsent(node2, new ArrayList<>());
14     graph.get(node1).add(node2);
15     graph.get(node2).add(node1);
16 }
```

3. Tampilkan isi graf menggunakan adjacency list

```
18 public void printGraph() {
19     System.out.println("Graf Awal (Adjacency List):");
20     for (String node : graph.keySet()) {
21         System.out.print(node + " -> ");
22         List<String> neighbors = graph.get(node);
23         System.out.println(String.join(", ", neighbors));
24     }
25     System.out.println();
26 }
```

4. Penelusuran graf secara DFS rekursif dan hindari simpul yang sudah dikunjungi

```
28 public void dfs(String start) {
29     Set<String> visited = new HashSet<>();
30     System.out.println("Penelusuran DFS:");
31     dfsHelper(start, visited);
32     System.out.println();
33 }
34
35 private void dfsHelper(String current, Set<String> visited) {
36     if (visited.contains(current)) return;
37     visited.add(current);
38     System.out.print(current + " ");
39     for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {
40         dfsHelper(neighbor, visited);
41     }
42 }
43 }
```

5. Penelusuran graf secara BFS iteratif menggunakan antrian

```
45 public void bfs(String start) {
46     Set<String> visited = new HashSet<>();
47     Queue<String> queue = new LinkedList<>();
48     queue.add(start);
49     visited.add(start);
50     System.out.println("Penelusuran BFS:");
51     while (!queue.isEmpty()) {
52         String current = queue.poll();
53         System.out.print(current + " ");
54         for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {
55             if (!visited.contains(neighbor)) {
56                 queue.add(neighbor);
57                 visited.add(neighbor);
58             }
59         }
60     }
61     System.out.println();
62 }
63 }
```

6. Membuat graf dengan 5 simpul dan menambahkan koneksi

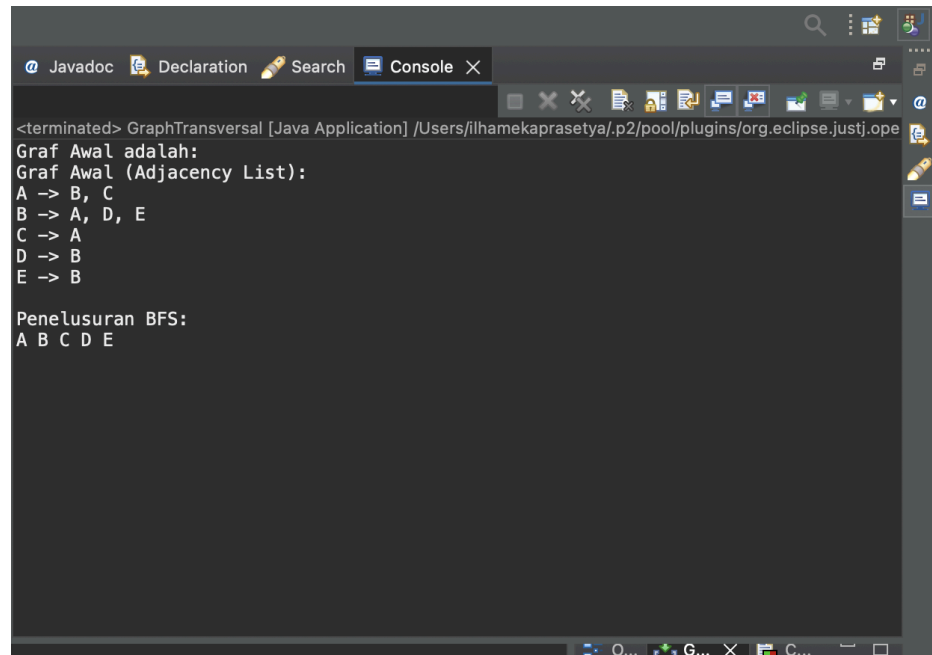
```
65 public static void main(String[] args) {
66     GraphTransversal graph = new GraphTransversal();
67
68     graph.addEdge("A", "B");
69     graph.addEdge("A", "C");
70     graph.addEdge("B", "D");
71     graph.addEdge("B", "E");
72 }
73 }
```

7. Tampilkan graf dan jalankan DFS dan BFS dari simpul A

```
72 graph.addEdge("B", "E");
73
74 System.out.println("Graf Awal adalah: ");
75 graph.printGraph();
76
77 |
78 graph.bfs("A");
79 }
80 }
81 }
```



Output:



```
<terminated> GraphTransversal [Java Application] /Users/ilhamkaprasetya/.p2/pool/plugins/org.eclipse.justj.ope
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran BFS:
A B C D E
```

D. KESIMPULAN

Pada praktikum Struktur Data pekan ke-9 ini, telah dipelajari dan diimplementasikan dua struktur data penting, yaitu **Binary Tree** dan **Graph**. Binary Tree digunakan untuk menyusun data secara hierarkis dengan hubungan induk-anak, dan mendukung penelusuran secara inorder, preorder, dan postorder. Melalui class Node, BTree, dan TreeMain, telah dibangun pohon biner secara bertahap dan mengeksplorasi bagaimana data dapat ditambahkan, dicari, serta dicetak dalam berbagai urutan.

Selanjutnya, juga mempelajari struktur **Graph** dengan representasi **adjacency list**, yang memungkinkan penyimpanan hubungan antar simpul secara fleksibel. Pada class GraphTransversal, diimplementasikan dua metode penelusuran graf, yaitu **Depth First Search (DFS)** dan **Breadth First Search (BFS)**. Kedua metode ini menunjukkan bagaimana data dalam graf dapat dijelajahi secara menyeluruh, baik dengan pendekatan rekursif (DFS) maupun menggunakan antrian (BFS).